PROJECT REPORT

EDUPULSE

ACADEMIC RISK ANALYTICS

Student Performance & Academic Risk Prediction

Big Data Analytics and Machine Learning Application

01 **Upload**

02 **Analyze**

03 **Predict**

04 **Review**

SUBMITTED BY	Vaishnavi P (814724243163)
	Vaishnavi S (814724243164)
	Saranya K (814724243132)
	Sheeba Princy D (814724243140)
COURSE	CCS334 – Big Data Analytics
INSTITUTION	SRM TRP Engineering College, Tiruchirappalli
REPORT DATE	November 2026

Data Preparation · Visualization · Machine Learning · Evaluation

ABSTRACT

Educational institutions generate substantial volumes of student information through attendance, assessment scores, study patterns, course progress and academic history. When these records are examined only through separate spreadsheets or manual summaries, it can be difficult to identify patterns across multiple indicators and recognize students who may benefit from timely academic support.

EduPulse is a student performance and academic risk analytics application developed to organize student-level records, explore performance indicators and demonstrate a supervised machine learning workflow. It accepts a CSV dataset or loads an included synthetic demonstration dataset. It validates required fields, handles selected missing values, removes duplicate student identifiers and prepares model features. A Random Forest classification model estimates an academic risk category using indicators such as study time, previous failures, absences and earlier-period grades.

The Streamlit interface presents dataset summaries, risk-category distributions, grade patterns, a searchable student explorer, an individual prediction interface and model evaluation outputs. Pandas and NumPy support data preparation, while Scikit-learn provides model training and evaluation. An optional PySpark script demonstrates grouped batch aggregation. The current implementation is a local application; it does not claim to deploy a Hadoop HDFS cluster or an Apache Hive warehouse.

The included demonstration data is synthetic and intended for software demonstration and testing only. Model scores must be captured from the actual execution environment and interpreted in the context of this dataset. Predictions are decision-support indicators, not definitive judgments about a student's ability or future.

Keywords: Big Data Analytics, Student Performance, Academic Risk, Data Preprocessing, Random Forest Classifier, Scikit-learn, Pandas, PySpark, Streamlit, Data Visualization.

Report completion note: Replace bracketed identity fields, add genuine application screenshots, and record metrics from your own run before final submission.

1. INTRODUCTION

Academic performance is influenced by several interacting factors. Assessment marks provide one view of progress, while attendance, study habits, previous academic difficulties and changes between assessment periods provide additional context. Reviewing these indicators together can help educators and students understand where support or further investigation may be useful.

In many learning environments, student records are stored in spreadsheets or separate departmental systems. Manual analysis can become time-consuming as the number of students, subjects and assessment periods increases. Repeated filtering and report preparation may also make it harder to compare trends consistently.

Data analytics offers a structured way to transform raw records into useful summaries. Data preparation can improve consistency, visualization can reveal patterns, and machine learning can estimate categories from historical examples. EduPulse combines these elements in a compact application that can be run locally and demonstrated using a CSV file.

1.1 Background

Student analytics commonly involves structured attributes such as student identifier, age, study-time category, previous failures, absences and assessment grades. These fields can be explored through descriptive statistics and visual comparisons. A classification model can be trained when a target category is available or derived from a defined academic outcome.

1.2 Project Motivation

- Provide a single interface for loading and exploring student performance data.
- Summarize academic indicators and identify records that may warrant closer review.
- Demonstrate a reproducible preprocessing and machine learning workflow.
- Present model evaluation information alongside predictions.
- Provide an extendable foundation for institution-approved data sources.

1.3 Intended Users

The prototype is intended for academic demonstrations and exploration by students, project evaluators and educators. Real-world use would require institutional approval, privacy safeguards, validation on representative data and human review.

2. PROBLEM STATEMENT, OBJECTIVES AND SCOPE

2.1 Problem Statement

Design and implement an interactive student analytics application that accepts structured academic records, performs data validation and preprocessing, visualizes performance indicators, and demonstrates machine learning-based classification of academic risk categories. The system should make the workflow understandable and provide outputs that can be reviewed by a human user.

2.2 Objectives

- Support CSV upload and a bundled synthetic demonstration dataset.
- Validate required columns, handle selected missing values and remove duplicate student identifiers.
- Display summary indicators, risk distribution, grade progression and selected comparisons.
- Provide filtering, search and downloadable student records.
- Train a Random Forest classifier and display category probabilities.
- Report held-out test performance, confusion matrix and feature importance where available.
- Provide an optional PySpark batch aggregation script.

2.3 Scope of the Current Prototype

Included in current build	Outside current build
Local CSV workflow and synthetic demo data	Live connection to a college ERP or student information system
Pandas preparation and Streamlit dashboard	Production deployment with authentication and role access
Scikit-learn classification and evaluation	Automated intervention or disciplinary decisions
Optional PySpark grouped aggregation	HDFS cluster, Hive metastore and Kafka streaming
Basic review suggestions	Validated institution-wide prediction service

The scope distinguishes implemented functionality from possible future enhancements. Features listed as future work must not be presented as completed modules.

3. TECHNOLOGY REVIEW AND ARCHITECTURAL COMPARISON

A student analytics workflow can be implemented using a range of tools. A spreadsheet may be sufficient for a small one-time summary, but repeated analysis and model evaluation benefit from a reusable data pipeline and application interface. The comparison below explains the design choices used in EduPulse.

Criterion	Manual / spreadsheet workflow	EduPulse prototype
Data loading	Manual file opening and repeated formatting	CSV upload or bundled demo dataset
Data preparation	Manual cleaning steps may vary	Programmatic validation and selected preprocessing
Exploration	Static tables and manual charts	Interactive charts and filters
Prediction	Manual interpretation or fixed rules	Supervised Random Forest classifier
Evaluation	Often reported separately	Held-out test evaluation and confusion matrix
Reproducibility	Depends on repeated manual steps	Code-based workflow and defined features
Scale	Convenient for small datasets	Local processing; PySpark script demonstrates grouped batch processing

3.1 Why a Classification Model?

The prototype groups final grade outcome into High risk, Medium risk and Low risk categories. A classification model learns patterns between selected input features and these labels. The category boundaries are part of the demonstration design and should be reviewed before applying the approach to a real institution.

3.2 Why Random Forest?

Random Forest combines predictions from multiple decision trees. It can model non-linear relationships and interactions among numeric features without requiring a linear relationship between each input and the target. It is used as a practical baseline for tabular classification. Its output probabilities are estimates, not guarantees.

3.3 Design Considerations

- Keep the target outcome out of model input features to reduce direct target leakage.
- Use a held-out test split to estimate performance on unseen records.
- Review class-wise evaluation because accuracy alone may hide minority-class errors.
- Treat synthetic-data results as a demonstration, not evidence of generalization.

4. TECHNOLOGY STACK AND REQUIREMENTS

Layer	Technology	Role in EduPulse
Programming	Python 3.10+	Application logic, data preparation and model workflow
Dashboard	Streamlit	Interactive local web interface
Data manipulation	Pandas, NumPy	CSV handling, validation and feature preparation
Machine learning	Scikit-learn	Random Forest classifier and evaluation
Visualization	Matplotlib / Streamlit charts	Charts for distributions and model interpretation
Big Data extension	PySpark (optional)	Grouped batch aggregation demonstration
Development	VS Code, PowerShell	Editing, environment setup and local execution
Version control	Git / GitHub (optional)	Source history and project sharing

4.1 Software Requirements

- Windows, Linux or macOS with a supported Python installation.
- Python virtual environment to isolate dependencies.
- Internet access during initial package installation.
- Modern browser to view the local Streamlit dashboard.
- Optional Java-compatible environment for the PySpark component.

4.2 Hardware Requirements

A standard personal computer with sufficient memory for Python, the dashboard and the selected CSV dataset is adequate for the demonstration dataset. Resource requirements increase with dataset size and model complexity. No multi-node cluster is required for the current local prototype.

4.3 Dependency Overview

The requirements file lists Streamlit, Pandas, NumPy, Scikit-learn, Matplotlib and PySpark. PySpark is included for the optional batch-processing script; the main dashboard does not require a running Spark cluster.

Implementation status: local Python application. Hadoop HDFS, Apache Hive, cloud deployment, real-time ingestion and institutional authentication are not implemented in this version.

5. SYSTEM DESIGN AND ARCHITECTURE

5.1 High-Level Architecture

The system is organized as a sequence of data input, preparation, analytics and presentation stages. The stages are separated conceptually so that data loading and preprocessing can be improved without redesigning the entire dashboard.

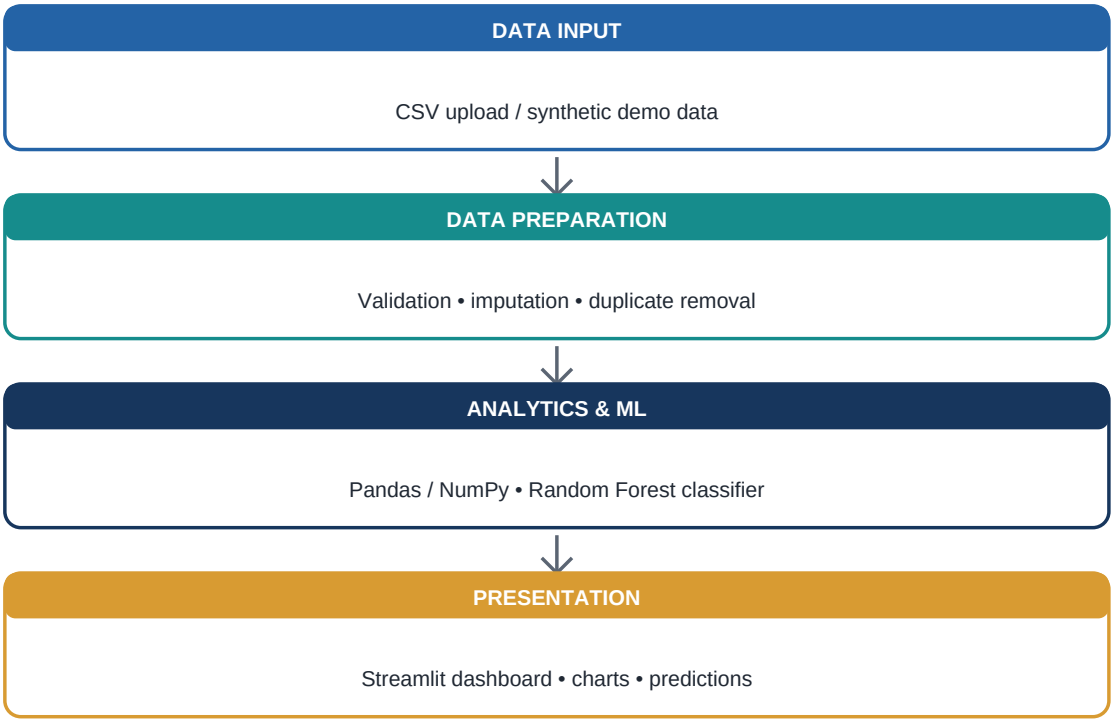


Figure 1: EduPulse high-level application architecture

5.2 Workflow Description

Stage	Description	Output
Input	Load uploaded CSV or demo file	Raw student records
Validation	Check required fields and data types	Validated dataframe or error
Preparation	Handle selected missing values; remove duplicate IDs	Cleaned dataframe
Feature selection	Separate predictors from target	Model-ready features and labels
Training	Fit Random Forest on training partition	Trained classifier
Evaluation	Predict held-out records and calculate metrics	Evaluation summary
Presentation	Render metrics, charts, filters and predictions	Interactive dashboard

6. DATASET DESCRIPTION AND PREPROCESSING

6.1 Dataset Overview

The bundled CSV contains 600 synthetic student records created for demonstration. It is not a real college dataset and should not be interpreted as representing any particular institution or student population. Users may upload a compatible CSV containing the required columns.

Attribute	Meaning	Use in application
student_id	Unique student record identifier	Search, display and duplicate removal
age	Student age in years	Model input feature
studytime	Encoded study-time category	Model input feature
failures	Previous academic failures	Model input and exploratory grouping
absences	Recorded absence count	Model input feature
G1	Earlier-period grade	Model input feature
G2	Later-period grade	Model input feature
G3	Final grade outcome	Derives target; excluded from predictors
gender	Demographic category in demo schema	Descriptive grouping where available

6.2 Target Category Definition

The application derives the demonstration risk label from final grade (G3): High risk for G3 below 10, Medium risk for G3 from 10 through 13, and Low risk for G3 of 14 or higher. These are prototype thresholds, not official academic standards. G3 is excluded from predictors to avoid directly giving the model the answer it is expected to predict.

6.3 Data Preparation

- Validate that required columns are present before analysis.
- Fill selected missing numeric values using the column median.
- Fill missing gender values with a neutral placeholder where used.
- Remove duplicate records by student_id.
- Separate input features from target category before training.

6.4 Data Quality Limitations

Synthetic data may contain simplified or artificial relationships. Real datasets may have different grading policies, missingness patterns, class imbalance and course structures. Cleaning rules and category thresholds must be reviewed with domain experts before real deployment.

7. DATA PROCESSING AND PYSPARK BATCH AGGREGATION

7.1 Processing in the Main Application

The main dashboard uses Pandas and NumPy to load and prepare tabular records locally. This is appropriate for the included demonstration dataset and supports fast iteration during development. The preprocessing stage produces a consistent dataframe for exploration and model training.

7.2 Optional PySpark Component

The project includes src/pyspark_batch.py as an optional demonstration of grouped aggregation using PySpark. It summarizes student records by a grouping attribute such as gender and writes the resulting summary to the outputs directory. This illustrates a distributed-data-processing API, but running it locally does not by itself establish a multi-node Spark cluster.

Operation	Purpose	Expected result
Read CSV	Load structured student records into a Spark DataFrame	Schema-aware input
Group records	Group by a categorical attribute	Groups such as gender
Aggregate	Calculate count and average values	Compact summary statistics
Write output	Save aggregated results to an output folder	Reusable summary dataset

7.3 Hadoop and Hive Status

HDFS ingestion commands, Hive external table definitions, physical partitioning and cluster replication are not part of the current implementation. They may be proposed as future extensions if the project is expanded to a configured Hadoop environment. This report does not claim HDFS or Hive execution results.

7.4 Potential Big Data Extension

For larger datasets, the pipeline could be extended with Spark-based distributed cleaning and aggregation, partitioned storage, a data warehouse layer and scheduled batch ingestion. A production architecture would also require access controls, audit logs, backups and monitoring.

8. MACHINE LEARNING PIPELINE IMPLEMENTATION

8.1 Learning Task

EduPulse frames academic risk estimation as a multi-class classification task. Each record is represented by selected academic indicators and assigned one of three target categories derived from the final grade in the demonstration dataset.

8.2 Input Features and Target

Role	Fields	Notes
Predictor features	age, studytime, failures, absences, G1, G2	Used to estimate category
Target label	Risk category derived from G3	High, Medium or Low risk
Excluded from predictors	G3	Excluded to prevent direct target leakage

8.3 Training and Testing

The implementation splits records into training and test subsets using a 75:25 ratio and stratification by target class where supported. The Random Forest model is fitted on the training subset and evaluated on the held-out test subset. A fixed random state supports repeatability for the same input data and software configuration.

8.4 Random Forest Classifier

A Random Forest classifier builds an ensemble of decision trees using sampled training records and feature subsets. The final class is determined by combining predictions from the trees. The implementation uses 180 estimators and configured reproducibility and class-splitting parameters in the application code.

8.5 Prediction Output

For an individual profile, the dashboard displays the predicted category and estimated class probabilities. The interface may also show general review suggestions associated with the predicted category. These suggestions are informational prompts, not individualized professional advice or automated interventions.

8.6 Model Limitations

- Synthetic-data performance does not establish performance on real student records.
- Predicted probabilities may not be calibrated and are not certainty.
- Historical data can encode bias or unequal access to educational resources.
- Human review and correction of inaccurate input data are essential.

9. APPLICATION MODULES AND USER INTERFACE

The Streamlit interface organizes the project into functional areas so a user can move from dataset overview to record-level exploration and predictive analysis.

Module	Main function	User value
Dataset loading	Choose demo dataset or upload CSV	Begins analysis using selected data
Data overview	Show record counts and summary indicators	Quick view of loaded data
Risk distribution	Visualize records in each category	Shows dataset distribution
Performance trends	Display grade progression and comparisons	Explore performance patterns
Student explorer	Filter/search records and download filtered data	Detailed record review
Risk prediction	Enter feature values and request estimate	Demonstrates individual inference
Model performance	Show test metrics, confusion matrix and feature importance	Transparent evaluation

9.1 Typical User Workflow

- Start Streamlit from the project directory.
- Choose the demo dataset or upload a CSV with required schema.
- Review the overview and risk distribution.
- Filter and inspect records in the student explorer.
- Enter an example profile in the prediction module.
- Review the predicted category and estimated probabilities.
- Inspect model performance before interpreting outputs.

9.2 Screenshot Placement Guide

Insert genuine screenshots captured from the running application. Recommended images: dashboard overview, risk distribution chart, student explorer, prediction result and model performance section. Do not use illustrative mockups as proof of actual execution.

10. MODEL EVALUATION AND RESULTS

10.1 Evaluation Approach

The model is evaluated on the held-out test subset. The dashboard reports metrics calculated from the actual run. Accuracy summarizes the fraction of correct predictions, while precision, recall and F1-score provide class-sensitive views. The confusion matrix shows which categories are confused with one another.

10.2 Metrics to Record

Metric	Interpretation	Actual run result
Accuracy	Overall proportion of correct classifications	[Enter dashboard value]
Precision	Correct positive predictions relative to all positive predictions	[Enter class-wise values]
Recall	Correctly identified records relative to all actual records	[Enter class-wise values]
F1-score	Harmonic mean of precision and recall	[Enter class-wise values]
Confusion matrix	Actual versus predicted category counts	[Insert screenshot / matrix]

10.3 Results Interpretation

No fixed accuracy or performance score is stated here because the final result must be taken from the application after execution with the selected dataset and installed package versions. Synthetic demo data confirms the application workflow but is not a substitute for validation on representative real-world data.

10.4 Recommended Result Capture

- Record the dataset name and number of rows used.
- Capture displayed accuracy and classification report.
- Save the confusion matrix image or screenshot.
- Note any class imbalance affecting interpretation.
- State whether the run used synthetic demo data or an approved real dataset.

Never copy example metrics from another report or invent values. Use results generated by your own execution.

11. SYSTEM TESTING AND VERIFICATION

Testing should verify the expected application workflow and its response to invalid or incomplete inputs. Mark each case Pass or Fail only after performing the test.

ID	Test scenario	Expected behaviour	Status
TC-01	Launch app	Dashboard opens without exception	[Pending]
TC-02	Load demo CSV	Dataset loads and overview appears	[Pending]
TC-03	Upload valid CSV	Records accepted and analyzed	[Pending]
TC-04	Upload CSV missing field	Validation identifies missing field(s)	[Pending]
TC-05	Duplicate student_id	Duplicates removed as designed	[Pending]
TC-06	Missing numeric values	Configured median imputation applied	[Pending]
TC-07	Search/filter records	Displayed records match criteria	[Pending]
TC-08	Download filtered records	CSV download is produced	[Pending]
TC-09	Run individual prediction	Category and probabilities displayed	[Pending]
TC-10	View model performance	Metrics and confusion matrix render	[Pending]

11.1 Execution Evidence

Attach screenshots of successful tests and retain error messages encountered during development. If a test fails, document the input, observed behaviour and correction. Do not mark a test as passed merely because a feature is present in source code.

11.2 Known Limitations

- Local prototype; no login, role management or institutional data connector.
- Model is trained from the currently loaded dataset in the application workflow.
- Demonstration data is synthetic and not a validated student population.
- PySpark batch script is optional and separate from the main dashboard.
- No HDFS, Hive, Kafka or production monitoring service is configured.

12. CONCLUSION AND FUTURE ENHANCEMENTS

12.1 Conclusion

EduPulse demonstrates an end-to-end local workflow for student performance exploration and academic risk classification. It brings together CSV ingestion, data validation, preprocessing, visualization and a supervised machine learning model within an interactive Streamlit application. The student explorer and prediction interface make it possible to inspect dataset-level patterns and an individual example profile, while the model performance section encourages evaluation alongside predictions.

The project also includes an optional PySpark aggregation script to introduce batch-processing concepts. The current build is limited to local processing and synthetic demonstration data. It does not implement a Hadoop HDFS cluster, Apache Hive warehouse or live institutional integration. Any reported model score must be taken from the actual run and interpreted cautiously.

12.2 Future Enhancements

- Integrate institution-approved academic data sources through secure interfaces.
- Introduce authentication, role-based access, audit trails and privacy-aware retention.
- Evaluate models using larger representative datasets with temporal and external validation.
- Compare Random Forest with Logistic Regression, Gradient Boosting and other baselines.
- Add explainability methods to help users understand feature influence.
- Extend batch processing with a configured Spark environment and partitioned storage.
- Add support workflows that keep educators in control of interventions.
- Monitor model drift and periodically review performance across cohorts.

12.3 Ethical and Responsible Use

Academic risk predictions can affect how students are perceived. The tool should be used only as a prompt for supportive review, never as the sole basis for grading, admission, scholarship, discipline or access decisions. Any deployment should include governance review, data minimization, fairness checks, transparency and a process for correcting inaccurate records.

EduPulse is a prototype for learning and demonstration. Its outputs require context, validation and human judgment.

REFERENCES AND APPENDIX

References

1. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

2. W. McKinney, “Data Structures for Statistical Computing in Python,” Proceedings of the 9th Python in Science Conference, 2010.

3. pandas development team, “pandas documentation,” https://pandas.pydata.org/docs/ (accessed November 2026).

4. Streamlit, “Streamlit documentation,” https://docs.streamlit.io/ (accessed November 2026).

5. Apache Spark, “PySpark documentation,” https://spark.apache.org/docs/latest/api/python/ (accessed November 2026).

6. L. Breiman, “Random Forests,” Machine Learning, vol. 45, pp. 5–32, 2001.

7. Python Software Foundation, “Python documentation,” https://docs.python.org/3/ (accessed November 2026).

Appendix A — Running the Application

1. Extract the ZIP and open the inner **EduPulse_Academic_Risk_Analytics** folder in VS Code.

2. Create and activate a virtual environment in PowerShell:

```
python -m venv .venv
.venv\Scripts\activate
```

3. Install dependencies and launch:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
streamlit run app.py
```

4. Open the local URL printed by Streamlit, usually http://localhost:8501.

5. For optional PySpark, configure a compatible Java environment and run `python src\pyspark_batch.py`.

Appendix B — Submission Checklist

- Replace guide, student names and register numbers on cover page.
- Add screenshots captured from the running application.
- Fill in actual evaluation metrics and test statuses.
- Confirm project title and scope with the project guide / department.
- Verify every report claim against the submitted source code.

— End of Report —